

# Requerimientos del Proyecto Final.

## Diseño de Experiencia de Usuario.

El centro de este trabajo es ¿cómo el usuario interactúa con el producto final?

|                                                                                                 |          |
|-------------------------------------------------------------------------------------------------|----------|
| <b>1. ¿Qué es el Ecosistema BrickScript?</b>                                                    | <b>2</b> |
| <b>2. Conceptos Fundamentales del Proyecto</b>                                                  | <b>3</b> |
| <b>3. Descripción de los Archivos del Proyecto</b>                                              | <b>5</b> |
| <b>4. Actividades:</b>                                                                          | <b>6</b> |
| Actividad 1: Conformación del Equipo de Trabajo                                                 | 6        |
| Actividad 2: Instalación y Validación del Entorno [10%]                                         | 6        |
| Actividad 3: Fase de Descubrimiento e Ideación [30%]                                            | 8        |
| Actividad 4: Estrategia de Diseño Inclusivo y Planificación de Pruebas [30%]                    | 12       |
| Actividad 5: Prototipado de Alta Fidelidad y Validación Final [30%]                             | 14       |
| Anexo 4.A. Ejemplo. Plan de Pruebas con Usuarios: Caso "Snake"                                  | 16       |
| Anexo 4.C. Ejemplo. Justificación UX: Diseño Inclusivo y Adaptación Narrativa                   | 18       |
| Anexo 4.D. Ejemplo. Adaptación para Limitaciones Visuales: Estrategias de Redundancia Sensorial | 20       |

# 1. ¿Qué es el Ecosistema BrickScript?

El ecosistema de **BrickScript** es una plataforma de **prototipado rápido** diseñada para facilitar el diseño, la experimentación y la evaluación de experiencias de juego mediante un Lenguaje de Dominio Específico (DSL). Este entorno permite que un diseñador defina las mecánicas, la estética y la interacción de un juego en un formato legible, transformándolo instantáneamente en un producto interactivo funcional para realizar pruebas de usuario.

Los componentes principales que permiten esta experimentación son:

- **Lenguaje de Definición (Archivos .brick):** Es la herramienta de diseño donde se configuran los elementos de la interfaz, el tamaño del área de juego y, fundamentalmente, el **mapeo de controles** y la respuesta del sistema ante las acciones del usuario.
- **Compilador (compiler.py):** Actúa como el motor de procesamiento que valida que las especificaciones de diseño sean coherentes antes de convertirlas en una experiencia jugable.
- **Representación Intermedia (Archivos .json):** Un formato puente que organiza las decisiones de diseño de manera estructurada, permitiendo que el motor las interprete sin demoras.
- **Motor de Ejecución o Runtime (runtime.py):** Es el entorno donde el diseño cobra vida. Se encarga de la **retroalimentación visual (feedback)**, la gestión del flujo del juego y la visualización de la puntuación en tiempo real.
- **Interfaz Gráfica y Control:** Utiliza la biblioteca Tkinter para ofrecer una representación visual estable y una gestión de eventos asíncrona, permitiendo que el usuario interactúe de forma fluida con el prototipo mediante el teclado.
- **Automatización (jugar.bat):** Un script que agiliza el ciclo de iteración, permitiendo que el diseñador realice un cambio en el diseño y lo pruebe inmediatamente con un solo comando.

Este ecosistema está optimizado para ser extremadamente ligero y portable, permitiendo su ejecución en hardware limitado sin dependencias externas, lo que facilita su uso en sesiones de guerrilla testing o laboratorios de usabilidad básicos. Siendo capaz de funcionar en sistemas antiguos como **Windows XP** y caber en un disquete 1.44Mb, utilizando exclusivamente **Python 2.7** sin necesidad de instalar librerías adicionales

## 2. Conceptos Fundamentales del Proyecto

Los siguientes términos son esenciales para comprender cómo el ecosistema permite iterar y evaluar la experiencia del usuario.

- **Lenguaje de Dominio Específico (DSL) como Herramienta de Diseño:** Es un lenguaje especializado que permite al diseñador "hablar" directamente con el sistema de juego. En lugar de programar lógica compleja, el DSL de BrickScript permite configurar la **usabilidad** del juego, definiendo cómo se ven las piezas y cómo responden los controles a la intuición del jugador.
- **Compilador (Validador de Diseño):** Actúa como el filtro que asegura que las decisiones de diseño sean coherentes. Su función técnica es traducir el código fuente, pero su valor en UX es garantizar que el prototipo esté listo para ser probado sin errores estructurales.
- **Gramática Formal (BNF) - El Manual de Estilo:** Define los límites de lo que el diseñador puede crear. Es el "contrato" que asegura que todos los juegos mantengan una consistencia en su estructura, facilitando que el usuario aprenda a jugar rápidamente gracias a patrones familiares.
- **Entorno de Ejecución (Runtime) y Retroalimentación (Feedback):** Es el motor donde ocurre la interacción. Desde la perspectiva de UX, el Runtime es responsable de la **retroalimentación inmediata**: gestionar que el dibujo en pantalla y el sistema de puntos respondan de forma gratificante a las acciones del usuario.
- **Representación Intermedia (Modelo de Datos):** Es el archivo JSON que organiza las decisiones estéticas y lógicas. Para el diseñador, es una estructura transparente que permite auditar qué parámetros de diseño están afectando la experiencia final.
- **Diferencia entre Error de Sintaxis y Error de Experiencia (Lógica):**
  - **Error de Sintaxis:** Un fallo estructural en el archivo `.brick` que impide que el prototipo inicie (ej. un paréntesis mal colocado). El sistema detiene la ejecución para evitar una experiencia rota.
  - **Error de Experiencia (Lógica):** El código es gramaticalmente correcto, pero la sensación de juego es deficiente. Por ejemplo, si una pieza cae demasiado rápido para la capacidad de reacción del usuario o si los colores elegidos confunden al jugador. Estos fallos se detectan mediante **pruebas de usuario**.
- **Iteración Rápida:** La capacidad de modificar un valor en el `.brick` y ver el cambio inmediato, permitiendo fallar rápido y barato durante la etapa de diseño.
- **Affordance Visual:** Cómo el diseño de las nuevas formas y colores en Tetris y Snake ayuda al usuario a entender intuitivamente qué objetos son beneficiosos o peligrosos.
- **Estado de Flujo (Flow):** El ajuste de la velocidad y la dificultad para mantener al jugador motivado, evitando tanto el aburrimiento como la ansiedad.
- **Guerrilla Testing (Pruebas de Guerrilla):** Es una técnica de evaluación de usabilidad rápida y de bajo costo que consiste en salir al entorno público para pedir a personas aleatorias que prueben el prototipo. En este proyecto, gracias a que el ecosistema es ligero y portable, los equipos pueden realizar estas pruebas de forma ágil, observando cómo un usuario desconocido interactúa con el juego de Snake o Tetris sin instrucciones previas.
- **A/B Testing (Pruebas A/B):** Consiste en crear dos versiones de un mismo diseño (Versión A y Versión B) con una sola diferencia variable para comparar cuál ofrece mejores resultados. Los estudiantes pueden usar el archivo `.brick` para generar

rápidamente dos variantes de un nivel (por ejemplo, una con colores fríos y otra con colores cálidos) y medir con qué versión los usuarios alcanzan una mayor puntuación o reportan menos fatiga visual.

- **Affordance Visual (Claridad de Uso):** Se refiere a las características de un objeto que sugieren por sí mismas cómo debe ser usado. En BrickScript, el diseño de las nuevas formas y colores debe comunicar intuitivamente su función: una fruta roja brillante debería "gritar" que es un premio, mientras que una nube gris debería sugerir peligro o un obstáculo, permitiendo que el usuario aprenda las reglas del juego mediante la observación visual directa.
- **Estado de Flujo (Flow):** Es el equilibrio psicológico donde el jugador está tan inmerso en la actividad que pierde la noción del tiempo. Los estudiantes deben ajustar variables como la velocidad de caída de las piezas en Tetris o el movimiento de la serpiente para evitar que el usuario caiga en el aburrimiento (si es muy fácil) o en la ansiedad (si es injustamente difícil).
- **Carga Cognitiva:** Es la cantidad de esfuerzo mental que el usuario debe realizar para procesar la información en pantalla. Al diseñar niveles como "Nyan Cat" una serpiente multicolor, el equipo debe analizar si el exceso de colores y obstáculos satura la atención del jugador, provocando errores que no se deben a la falta de habilidad, sino a un diseño de interfaz confuso.

En este curso, los equipos no actúan como programadores de videojuegos, sino como **Arquitectos de Experiencia**. Un sistema de diseño es un organismo vivo que debe adaptarse a las necesidades del usuario final.

- **Evolución de la Interfaz:** Originalmente, BrickScript ofrecía una interacción muy limitada. Para mejorar la **satisfacción del usuario**, el equipo debe intervenir el sistema para permitir elementos como el color, niveles de dificultad o "Power-Ups" que reduzcan la frustración del jugador.
- **Construcción de Primitivas de Diseño:** Modificar el código de Python no es un fin en sí mismo, sino un medio para crear una **infraestructura de prototipado** más potente. Una vez que el sistema permite etiquetas de color o probabilidades, el diseñador puede iterar cientos de variaciones estéticas sin tocar el código base.
- **Ciclo de Iteración UX:** El objetivo es experimentar el ciclo completo: desde definir una regla de interacción en la gramática, hasta observar cómo esa regla afecta la emoción del jugador en el Runtime. Al abrir la "caja negra" del proyecto, el lenguaje se convierte en un ecosistema adaptable a cualquier necesidad de diseño.

**Meta Final: Consistencia y Retrocompatibilidad.** Un diseño exitoso es aquel que evoluciona sin destruir el **modelo mental** previo del usuario. El sistema debe ser capaz de ejecutar las versiones más avanzadas sin perder la capacidad de ofrecer las experiencias clásicas y sencillas originales.

### 3. Descripción de los Archivos del Proyecto

Es fundamental comprender la función de los componentes que integran el ecosistema de BrickScript. A continuación, se detalla el contenido de cada archivo, estos archivos están en “*RetroBrik Windows Version - Proyecto Base 2026-1s.zip*” descargable del curso:

- **compiler.py**: Este archivo constituye el núcleo del sistema de traducción, encargándose de transformar el código fuente escrito en lenguaje BrickScript en un formato estructurado de datos JSON mediante un analizador léxico y un parser de descenso recursivo.
- **runtime.py**: Actúa como el motor de ejecución del juego y utiliza la biblioteca gráfica Tkinter para interpretar los archivos JSON generados, gestionando la lógica visual, el movimiento de los objetos y la detección de eventos en tiempo real.
- **jugar.bat**: Es un script de automatización diseñado para sistemas Windows que simplifica el flujo de trabajo del usuario al coordinar la ejecución secuencial del compilador y el motor de juego con un solo comando.
- **README.txt**: Proporciona la documentación esencial del proyecto, detallando los objetivos de BrickScript, las instrucciones de instalación y una guía rápida sobre la sintaxis necesaria para programar los juegos de ejemplo.
- **INSTALL.txt**: Contiene los requisitos técnicos específicos para el despliegue del software, advirtiendo sobre la necesidad estricta de utilizar Python 2.7 y ofreciendo soluciones a errores comunes de configuración del sistema.
- **snake.brick**: Representa un ejemplo práctico de código fuente que define la lógica, los colores y las reglas de interacción del juego de la serpiente utilizando la sintaxis específica del lenguaje BrickScript.
- **tetris.brick**: Este archivo de ejemplo ilustra cómo definir formas complejas y rotaciones de piezas dentro del lenguaje, permitiendo a los estudiantes observar la implementación de la lógica clásica del juego Tetris.
- **snake.bnf**: Establece la especificación formal de la gramática para el juego Snake, definiendo las reglas sintácticas que el compilador debe reconocer para validar las acciones y eventos permitidos dentro de ese contexto.
- **tetris.bnf**: Define formalmente la estructura gramatical requerida para el juego de Tetris, sirviendo como mapa de referencia para entender cómo el lenguaje organiza las definiciones de piezas, cuadrículas y eventos de teclado.
- **snake.json / tetris.json**: Estos archivos son el resultado del proceso de compilación y contienen una representación intermedia de los juegos en un formato que el motor de ejecución puede procesar de manera eficiente.

## 4. Actividades:

### Actividad 1: Conformación del Equipo de Trabajo

| # | Nombre Completo               | Doc. Id.   | Correo Electrónico                                           |
|---|-------------------------------|------------|--------------------------------------------------------------|
| 1 | Alejandro Feria Gonzalez      | 1011510542 | <a href="mailto:aferiag@unal.edu.co">aferiag@unal.edu.co</a> |
| 2 | Juan Fernando Quintero Perez  | 1035971216 | juquinterope@unal.edu.co                                     |
| 3 | Abraham David Miguel Cardenas | 1038095269 | amiguel@unal.edu.co                                          |
| 4 |                               |            |                                                              |
| 5 |                               |            |                                                              |

### Actividad 2: Instalación y Validación del Entorno [10%]

Esta actividad consiste en la puesta en marcha del proyecto y la ejecución de los juegos de ejemplo suministrados en el proyecto base. Está en “*RetroBrik Windows Version - Proyecto Base 2026-1s.zip*” descargable del curso.

#### A. Requisitos Previos

Antes de comenzar, asegúrese de cumplir con las especificaciones técnicas del proyecto (ver archivo INSTALL.txt):

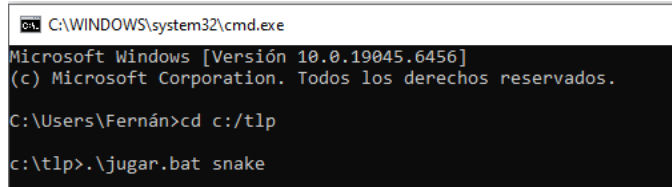
- **Intérprete de Python:** Es obligatorio el uso de **Python 2.7**. Tenga en cuenta que el código **no es compatible** con Python 3 o versiones superiores.
- **Sistema Operativo:** El sistema está diseñado principalmente para **Windows XP** o superior, aunque puede ejecutarse en otros entornos que soporten Python 2.7 y la librería **Tkinter**.
- **Dependencias:** Aunque el proyecto solo utiliza módulos de la librería estándar de Python, los estudiantes deben realizar una verificación activa de las dependencias según su plataforma:
  - **En Windows:** Dependiendo de la versión de Python 2.7 instalada (por ejemplo, si se usó un instalador personalizado o minimalista), podría ser necesario reinstalar o activar el componente **Tcl/Tk** para que **Tkinter** funcione correctamente.
  - **En Linux:** Por lo general, **Tkinter** no se instala por defecto con el paquete base de Python. Es probable que el estudiante deba instalarlo manualmente mediante el gestor de paquetes (por ejemplo, `sudo apt-get install python-tk`).
  - **Verificación:** Se recomienda ejecutar el comando `python -c "import Tkinter; print 'Tkinter listo'"` en la terminal antes de iniciar el trabajo para confirmar la disponibilidad de la interfaz gráfica.

```
C:\Python27>python -c "import Tkinter; print 'Tkinter listo'"
Tkinter listo
```

## B. Instrucciones de Ejecución

Para ejecutar los juegos, siga estos pasos desde la consola de comandos (cmd.exe) (ver archivo README.txt):

1. Navegue hasta la carpeta raíz del proyecto, por ejemplo: `cd C:\tpl`.
2. Utilice el script de automatización `jugar.bat` seguido del nombre del juego.
  - **Para probar Snake:** escriba `.\jugar snake`.

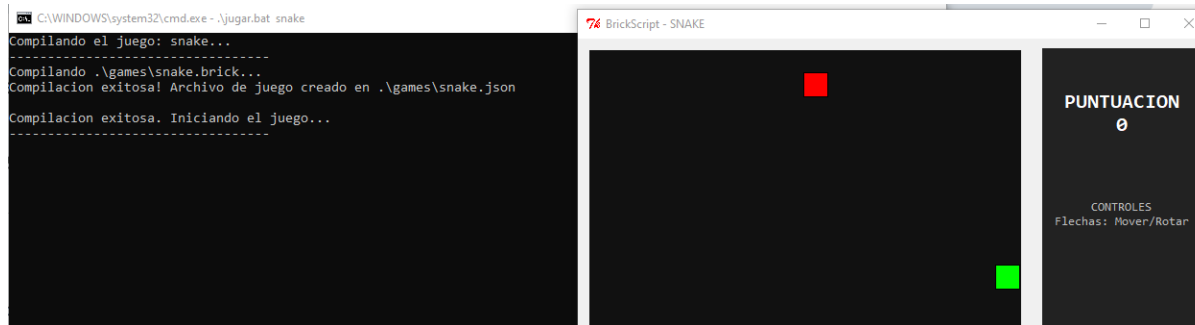


```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Versión 10.0.19045.6456]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Fernán>cd c:\tpl

c:\tpl>.\jugar.bat snake
```

- Luego Enter.



- **Para probar Tetris:** escriba `.\jugar tetris`

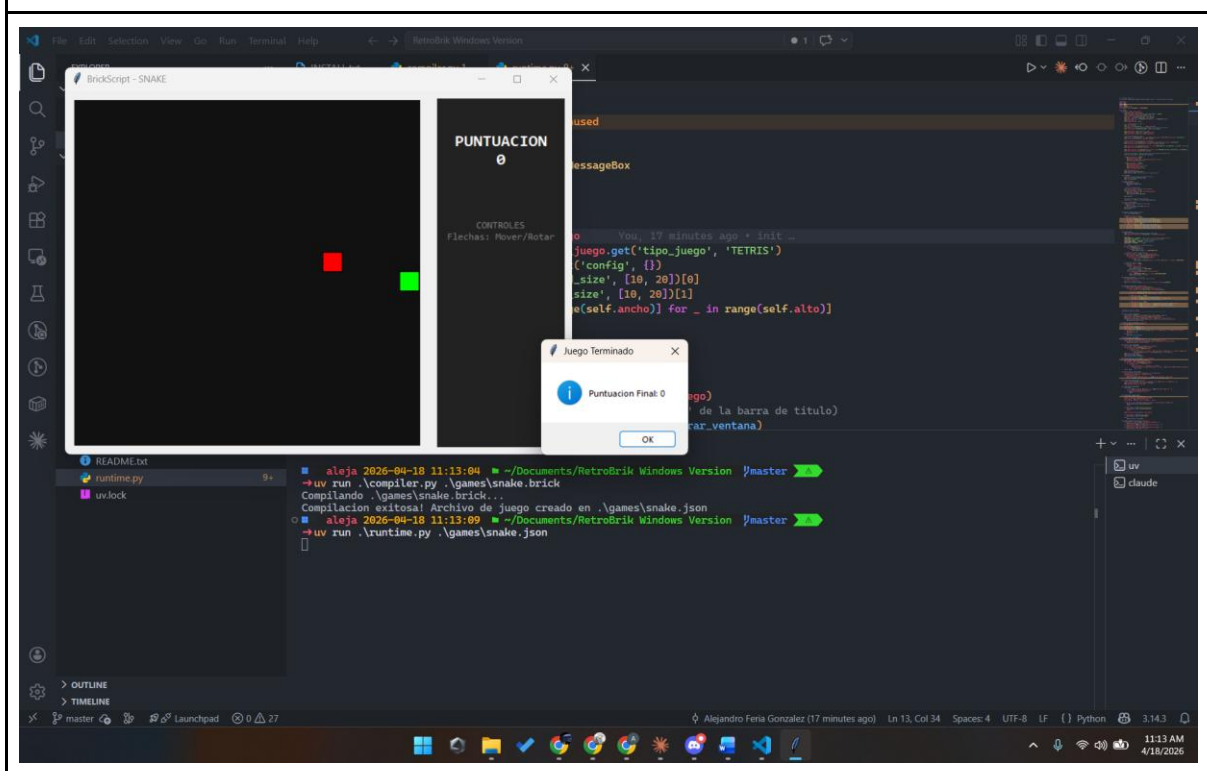
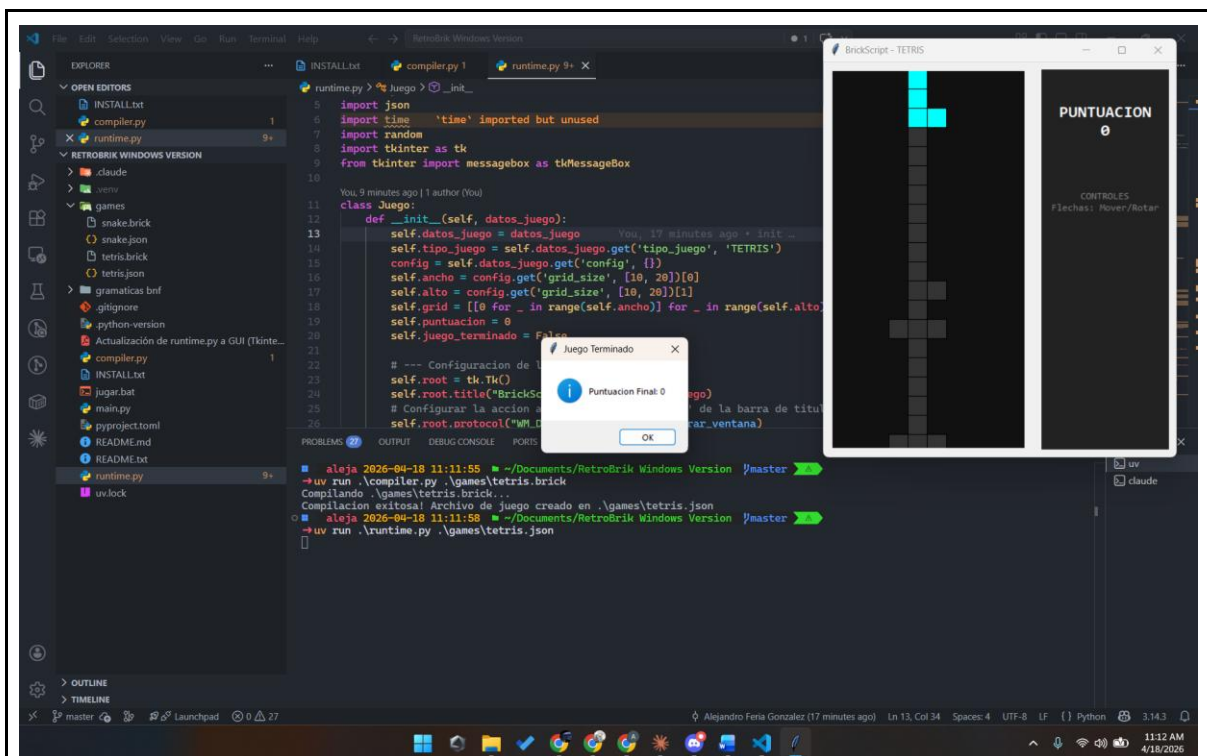
3. El sistema realizará automáticamente la compilación del archivo `.brick` a `.json` y, si no hay errores, iniciará el motor de juego (`runtime.py`) con la interfaz gráfica.

## C. Entregable Actividad 2

Como prueba de la correcta ejecución, cada Equipo de Trabajo debe capturar una toma de pantalla (screenshot) que cumpla con los siguientes requisitos obligatorios:

- **Ventana del Juego:** Debe verse la interfaz gráfica del juego (Snake o Tetris) en pleno funcionamiento.
- **Consola de Comandos:** Debe apreciarse la ventana del terminal donde se vea el comando ejecutado y el mensaje "Compilación exitosa".
- **Reloj del Sistema:** Es estrictamente necesario que en la captura se aprecie claramente la **fecha y hora** actual en la barra de tareas o el reloj del sistema operativo (Windows, Linux o macOS).

**Nota:** Las capturas donde no sea legible la fecha y hora del sistema no serán tomadas como válidas para la calificación de esta actividad.



### Actividad 3: Fase de Descubrimiento e Ideación [30%]

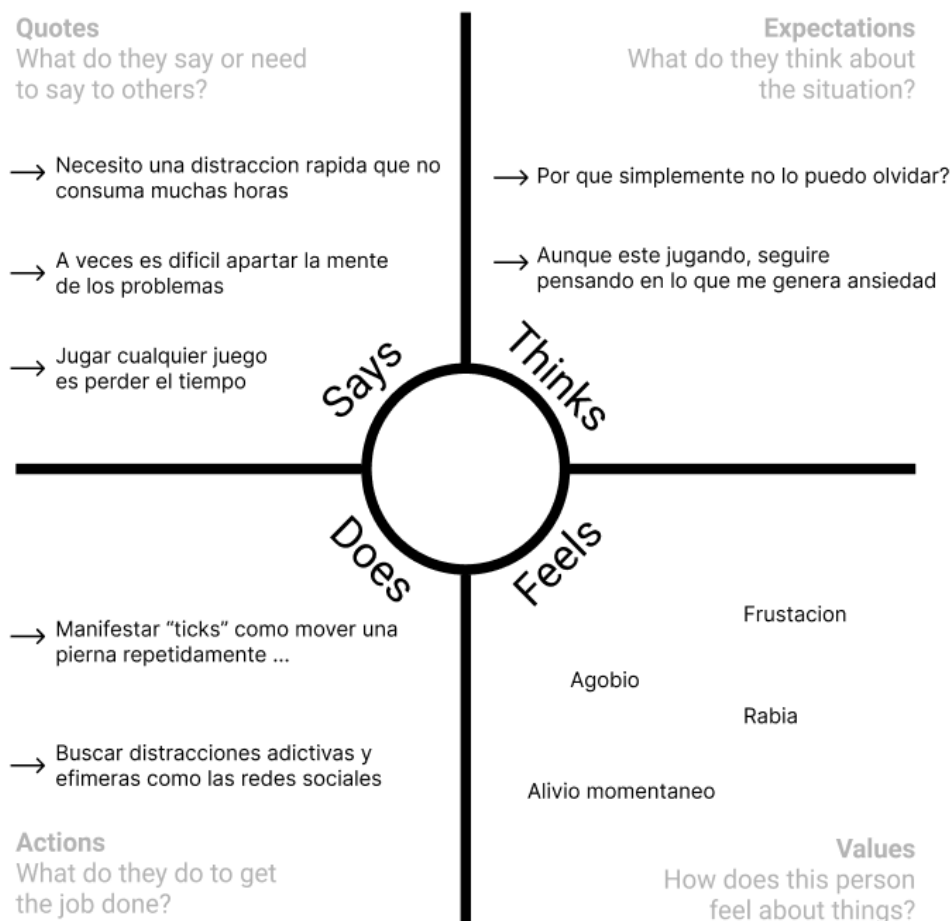
Enfoque: Investigación, empatía y definición del propósito social del juego. Objetivo: Analizar cómo una versión modificada de Tetris o Snake puede ayudar a un usuario específico

enfrentando un desafío particular (estrés, ludopatía, productividad, salud mental, discapacidad o condición neurológica).

**NO Olvides que para poder desarrollar el trabajo debes ejecutar tetris y snake**, y estos puntos se desarrollan bajo ese contexto de ejecución, es decir quien va a usar o jugar tetris y snake. **Cuando un artefacto tenga links complementarios**, por favor primero consúltalos y luego haces la tarea, esto te ayudará mucho.

## A. Empatía y Perfil de Usuario

- **Mapa de Empatía:** Un cuadrante que detalle qué ve, oye, piensa, siente, dice y hace el usuario objetivo ante el reto planteado.
  - <https://www.uxdesigninstitute.com/blog/what-is-an-empathy-map/>
  - <https://asana.com/es/resources/empathy-map-template>
  - <https://innokabi.com/mapa-de-empatia-zoom-en-tu-segmento-de-cliente/>
  - <https://www.nngroup.com/articles/empathy-mapping/>



Fuentes: Holmes et al. (2009, Psychological Science); Skalski et al. (2011)

- **Proto-persona:** Creación de un personaje ficticio (ej. "Andrés, 22 años, estudiante con ansiedad diagnóstica") que incluya sus necesidades tecnológicas, frustraciones con las interfaces actuales y objetivos al usar el juego.
  - o <https://outwitly.com/resources/what-is-a-proto-persona/>
  - o <https://www.aluracursos.com/blog/guia-practico-de-proto-personas-y-personas>
  - o <https://www.ridgeway.com/blog/experience-design-ux/what-are-personas>

Nombre: Andrés.

Edad: 22 años.

Ocupación: Estudiante universitario de Ingeniería.

Condición: Ansiedad generalizada diagnosticada.

- Biografía

Andrés utiliza la tecnología como su principal herramienta de estudio, pero también es su mayor fuente de distracción. Cuando se enfrenta a fechas de entrega, se queda bloqueado. Busca una herramienta que le permita pausar el ruido mental sin caer en las redes sociales.

- Frustraciones

Sobrecarga de dopamina: Le frustran los juegos que usan sonidos estridentes o luces parpadeantes tras cada nivel (efecto tragamonedas).

Penalización por error: Le genera ansiedad perder "vidas" o ser castigado por el sistema cuando comete un error en el juego.

Falta de propósito: Siente culpa al usar aplicaciones que no percibe como beneficiosas para su estado emocional.

Fuente: La investigación de Lumsden et al. (2016) señala que la "gamificación agresiva" puede ser contraproducente en intervenciones de salud mental, aumentando el estrés en lugar de reducirlo.

B. Investigación Temática o Diseño Basado en Evidencia. (Contexto en el cual se va a usar la aplicación o software)

- **Marco de Referencia:** Una revisión breve (máximo 1 - una página, con referencias reales) sobre contexto de uso de tetris y snake. (ej. "Uso de Tetris en la reducción de flashbacks por estrés postraumático" o "Snake como herramienta de coordinación motriz en pacientes con epilepsia focal").
- **Fuentes:** Es obligatorio citar al menos 3 fuentes académicas de publicaciones indexadas (ej. ScienceDirect), de las no indexadas es válido usar arXiv.

### 1. Tetris y la interrupción de memorias nocivas

Tetris y juegos similares han sido estudiados con sentidos terapéuticos, principalmente como “remedio” para síntomas de estrés postraumáticos, siendo útil al usar áreas del cerebro encargadas de gestionar la memoria visoespacial; mismos recursos usados a la hora de consolidar imágenes sensoriales traumáticas.

Fuente: Holmes, E. A., et al. (2009). "Can Playing the Computer Game 'Tetris' Reduce the Build-Up of Flashbacks for Trauma? A Quasi-Experimental Study of Distactor Tasks After Exposure to Traumatic Video Content"

### 2. Snake y la mejora motriz

Snake se basa en la planificación y la coordinación ojo-mano por ello se ha evaluado su uso en la mejora general de la coordinación motriz y más específicamente sobre pacientes con afectaciones neurológicas, recibiendo progreso en la corteza motora y prefrontal.

Fuente: Lumsden, J., et al. (2016). "Gamification of Cognitive Assessment and Cognitive Training: A Systematic Review"

### 3. Efectos sobre el bienestar y la paciencia

Ambos juegos han demostrado ser eficaces para inducir el estado de Flow (fluidez), un estado psicológico donde el usuario está tan inmerso en la tarea que el sentido del tiempo y la autocritica desaparecen. Esto es crucial en entornos hospitalarios o de alto estrés.

Fuente: Rodriguez, A., et al. (2021). "The Effects of Brief Play of Video Games on Stress Recovery"

## C. Accesibilidad y Usabilidad

- **Checklist de Accesibilidad:** Evaluación de una Lista de principios (basados en WCAG consultar <https://www.w3.org/TR/WCAG21/>) que el equipo debe revisar en tetris y snake (ej. uso de alto contraste, patrones visuales para daltónicos o tiempos de reacción extendidos).
- Deben consultar la lista de los criterios de éxito en WCAG 2.1 y evaluar cada uno de ellos. El equipo tiene la libertad de seleccionar la escala que desee. Se recomienda usar una escala de Likert.

**Principio perceptible:** La información y los componentes de la interfaz de usuario deben ser presentables a los usuarios en formas que puedan percibir.

| <b>Criterio WCAG 2.1</b>           | <b>Descripción</b>                                                                 | <b>Escala (1-5)</b> |
|------------------------------------|------------------------------------------------------------------------------------|---------------------|
| <b>1.1.1 Contenido no textual</b>  | ¿Tienen las piezas alternativas sonoras o patrones para quienes no ven color?      | 1                   |
| <b>1.4.1 Uso del color</b>         | ¿Se depende solo del color para diferenciar piezas?                                | 1                   |
| <b>1.4.3 Contraste mínimo</b>      | ¿La relación de contraste entre las piezas y el fondo es de al menos 4.5:1?        | 2                   |
| <b>1.4.11 Contraste no textual</b> | ¿Los bordes de las piezas de Snake y los límites del muro son claramente visibles? | 5                   |

**Principio operable:** Los componentes de la interfaz de usuario y la navegación deben ser operables.

| <b>Criterio WCAG 2.1</b>         | <b>Descripción</b>                                                                      | <b>Escala (1-5)</b> |
|----------------------------------|-----------------------------------------------------------------------------------------|---------------------|
| <b>2.1.1 Teclado</b>             | ¿Se puede jugar la totalidad de la partida usando solo el teclado?                      | 5                   |
| <b>2.2.1 Tiempo ajustable</b>    | ¿Puede el usuario pausar o ralentizar la velocidad de caída/movimiento?                 | 1                   |
| <b>2.3.1 Umbral de destellos</b> | ¿El juego evita destellos o animaciones rápidas que puedan causar ataques de epilepsia? | 5                   |
| <b>2.4.7 Foco visible</b>        | ¿Es claro qué botón o menú está seleccionado en la pantalla de inicio?                  | 3                   |

**Principio Comprensible:** La información y el manejo de la interfaz de usuario deben ser comprensibles.

| Criterio WCAG 2.1               | Descripción                                                    | Escala (1-5) |
|---------------------------------|----------------------------------------------------------------|--------------|
| 3.2.1 Al recibir el foco        | ¿Ocurren cambios inesperados de contexto al mover una pieza?   | 5            |
| 3.3.2 Etiquetas o instrucciones | ¿Hay una guía clara de cómo jugar antes de iniciar la partida? | 3            |

**Principio Robusto:** El contenido debe ser lo suficientemente robusto como para ser interpretado por una amplia variedad de agentes de usuario.

| Criterio WCAG 2.1            | Descripción                                                                                   | Escala (1-5) |
|------------------------------|-----------------------------------------------------------------------------------------------|--------------|
| 4.1.2 Nombre, función, valor | ¿Los lectores de pantalla pueden identificar el estado del juego (ej. "¿Nivel 2", "Pausado")? | 5            |

#### D. Guerrilla Testing (Pruebas de Guerrilla).

- Es deseable hacerlo con usuarios desconocidos, de no ser posible, lo pueden hacer los integrantes del equipo. El propósito es encontrar todas las incomodidades, falencias, perezas, inconsistencias, etc. en concreto es encontrar todas las falencias a nivel de funcionamiento. Tener en cuenta la diferencia entre un "bug" (el programa se cerró) y una "falla de usabilidad" (el usuario no entendió cómo rotar la pieza)
  - <https://ixdf.org/literature/topics/guerrilla-testing>
  - <https://usabilitygeek.com/guerrilla-usability-testing-how-to/>
  - <https://maze.co/guides/usability-testing/guerrilla/>

| ID  | Componente              | Acción                                                | Resultado Esperado                                                      | Resultado Observado                                                          | Tipo |
|-----|-------------------------|-------------------------------------------------------|-------------------------------------------------------------------------|------------------------------------------------------------------------------|------|
| Q01 | Mecánica (Tetris)       | El usuario completa una línea horizontal.             | La línea desaparece y se suma puntaje.                                  | Acorde                                                                       | Bug  |
| Q02 | Control (Snake)         | El usuario presiona la dirección opuesta a la actual. | La serpiente ignora el comando (evita el suicidio instantáneo).         | Acorde                                                                       | Bug  |
| Q03 | Interfaz                | El usuario con ansiedad intenta pausar el juego.      | El botón de pausa es visible y detiene el estímulo visual de inmediato. | El usuario no encuentra el botón rápido o el juego sigue corriendo de fondo. | UX   |
| Q04 | Mecánica (Tetris)       | Rotación de pieza contra una pared (Wall Kick).       | La pieza debe rotar ajustándose al espacio disponible.                  | La pieza se queda bloqueada.                                                 | Bug  |
| Q05 | Accesibilidad           | Uso de patrones visuales (Daltónicos).                | El usuario diferencia las piezas sin necesidad de ver colores.          | Modo de colores único                                                        | UX   |
| Q06 | Lógica (Snake)          | Crecimiento tras comer el "ítem de beneficio".        | El cuerpo aumenta un segmento de forma fluida.                          | Acorde                                                                       | Bug  |
| Q07 | Maniobrabilidad (Snake) | Mover la serpiente en muchas direcciones a la vez.    | La serpiente gira de forma fluida y acorde.                             | En un movimiento específico la serpiente camina por ella misma y muere.      | Bug  |

Como mejora de los errores: Q03, Q04, Q05, Q07; en general implican desarrollos extras y no simples “ajustes”. Por ejemplo, los Q03 y Q05 se podrían solucionar con un menú fijo e intuitivo que administre configuraciones y controles en donde incluir las acciones faltantes: pausar el juego o modo daltónico; los problemas Q04 y Q07 son de tipo bug, por lo que son fallos lógicos que habría que encontrar y arreglar dentro del código.

## E. Auditoría de Usabilidad

Ahora se debe realizar una **Auditoría de Usabilidad** sobre la versión base del proyecto utilizando este derrotero, cada respuesta se debe responder detallada y exhaustivamente:

- **Evaluación Heurística Inicial** (estas preguntas son base, **agrega al menos 2 preguntas relacionadas con cada criterio.**)
  - **Visibilidad del Estado:**
    - ¿El usuario sabe en todo momento su puntuación y si el juego está pausado?
      - La puntuación es visible en todo momento, pero no existe un botón claro para pausar el juego.
    - ¿Ante una acción inesperada fuera de la ventana del juego (ej. Un click fuera) El juego se pausa automáticamente y tiene un estado claro de reanudar?
      - Esta funcionalidad existe parcialmente, al haber actividad fuera de la ventana el juego se pausa pero no es visible una vista clara de pausa, en su lugar el juego se reanuda solo al volver a la ventana.
  - **Consistencia:**
    - ¿Los colores de las piezas en Tetris ayudan a identificarlas o generan confusión visual?
      - No, todas las piezas tienen el mismo color.
    - ¿La tipografía mantiene la misma estética?
      - Si, la puntuación y sección de como jugar lo hacen
    - ¿En ambos juegos la mecánica de niveles, velocidad y sistema de juego se mantiene como se espera?.
      - Si, a medida que el juego avanza este mantiene consistencia en su forma de juego.
  - **Prevención de Errores:**
    - ¿El sistema de rotación es predecible o causa que la pieza "salte" y el usuario pierda por un error del software?
      - Parcialmente, no todas las rotaciones de una misma pieza están implementadas.
    - ¿En Snake existe un breve margen de tiempo cuando se presionan teclas contrarias para evitar colisión con el propio cuerpo?
      - No, este es un bug identificado en el juego.
    - ¿En ambos juegos, los puntajes siguen avanzando coherentemente?
      - Si, los puntos mantienen coherencia.

- **Análisis de Carga Cognitiva** (estas preguntas son base, **agrega al menos 2 preguntas relacionadas con cada criterio.**)
  - **Velocidad vs. Frustración:**
    - Evaluar si el "tick" inicial en `runtime.py` permite al usuario procesar la información o si la curva de aprendizaje es demasiado empinada desde el segundo 1.
      - Se evidencia una dificultad gradual y aceptable.
    - ¿El usuario es capaz de mantener una conversación mientras juega?
      - Si, esto indica que no utiliza toda la capacidad cognitiva y es ideal para relajarse.
  - **Ruido Visual:**
    - Identificar si el diseño actual del "grid" distrae de las piezas activas.
      - No, hay buena diferenciación entre el grid de fondo y los bloques de los juegos.
    - ¿Existen mensajes que se atraviesen en el campo de juego al obtener o alcanzar hitos?
      - No, todo el juego transcurre de forma fluida sin interrupciones.
- **Mapeo de Controles (Interacción)** (estas preguntas son base, **agrega al menos 2 preguntas relacionadas con cada criterio.**)
  - **Intuición:**
    - ¿Es natural usar la flecha arriba para rotar? Realizar una prueba de guerrilla con 2 personas ajenas al equipo y anotar si intentaron usar otra tecla por instinto.
      - Si fue natural, la explicación fue que en este tipo de juegos retos se asocian a las flechas como comandos para jugar, tal vez por su sencillez.
    - ¿El intentar bajar la pieza mas rapido es intuitivo usar la flecha hacia abajo?
      - Si también lo es, se asocio fácilmente con llevar la pieza hacia abajo.
    - ¿Es sencillo identificar el objetivo de Snake?
      - Las respuestas fueron divididas, al ser totalmente cubico algunas personas no identificaron que se trataba del juego Snake, las que si lo hacían entendían el objetivo inmediatamente, en cualquier caso se dependía del conocimiento previo del juego.
  - **Feedback:**
    - ¿El juego emite una señal clara (visual o de cambio de ritmo) cuando el usuario está a punto de perder?
      - No, independientemente del flujo del juego, todo continua monotonamente.

- ¿Existe una identificación clara ya sea visual o sonora que ayude a identificar si una pieza encaja o la serpiente comió una manzana?
    - No, el juego sigue manteniendo la monotonía.
  - ¿Existen alertas auditivas o visuales que alerten cambios de ritmo?
    - No, aunque en general ambos juegos no tienen cambios de velocidades evidentes.
- **Diagnóstico de Errores** (estas preguntas son base, **agrega al menos 2 preguntas relacionadas con cada criterio.**)
  - **Mensajes del Sistema:**
    - Si el usuario comete un error de sintaxis al intentar personalizar su juego, ¿el mensaje de `compiler.py` le ayuda a entender qué pasó o es demasiado técnico y frustrante?
      - Es claro desde un conocimiento previo en Python.
    - ¿El lenguaje utilizado en las alertas evita códigos de error crípticos (ej. "Error 0x004") y propone una solución accionable para el usuario no técnico?
      - Si, mapea el error para que este en lenguaje natural.
    - ¿Se indica la línea exacta del error en caso de que estos ocurran?
      - Si, la línea viene asociada con el error de compilación.
  - **Tiempo de Respuesta:**
    - ¿El usuario siente que el sistema reacciona lo suficientemente rápido a sus pulsaciones de teclas o hay un retraso percibido?
      - Hay una latencia adecuada.
    - ¿Se identifican caídas de fps cuando existen muchos elementos en el nivel?
      - No, el juego continúa fluido.

## F. Pitch Narrativo y Estructura de una Nueva Versión

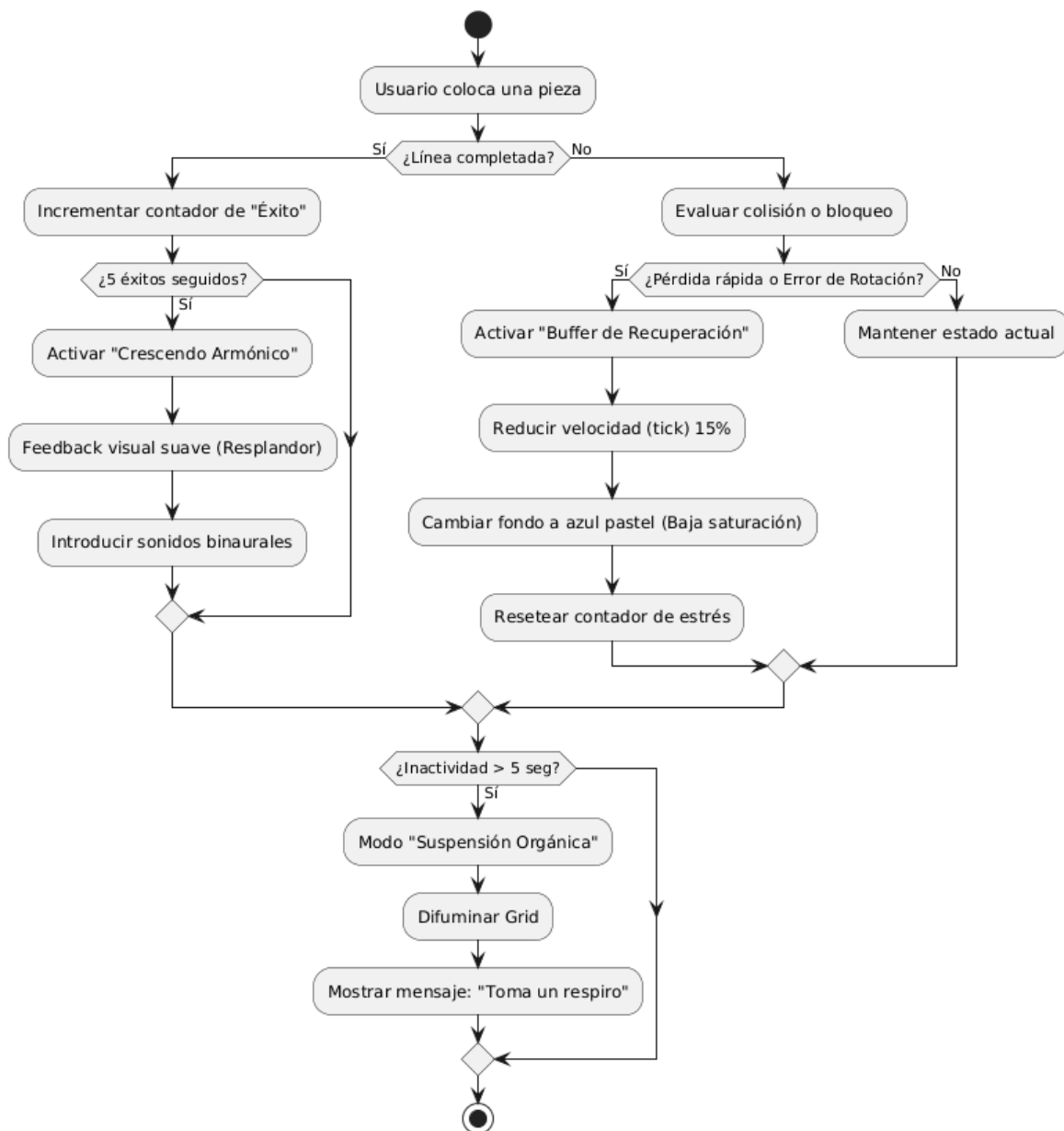
- **Pitch de 1 Párrafo:** Una descripción convincente de por qué se requiere otra versión del juego, evidentemente diferente. No es "un Tetris de colores", es "una experiencia de regulación emocional a través del orden geométrico". **Tetris: Arquitectura del Sosiego**

**Tetris:** Más que un puzzle de encaje, se requiere una versión beneficiosa cognitivamente diseñada para interrumpir ciclos de rumiación y estrés postraumático a través del orden geométrico. Al transformar el caos visual en estructuras armónicas bajo un flujo constante y controlado, el juego actúa como un regulador de la memoria de trabajo, permitiendo al usuario transitar de un estado de alerta ansiosa a uno de autoeficacia y calma mental.

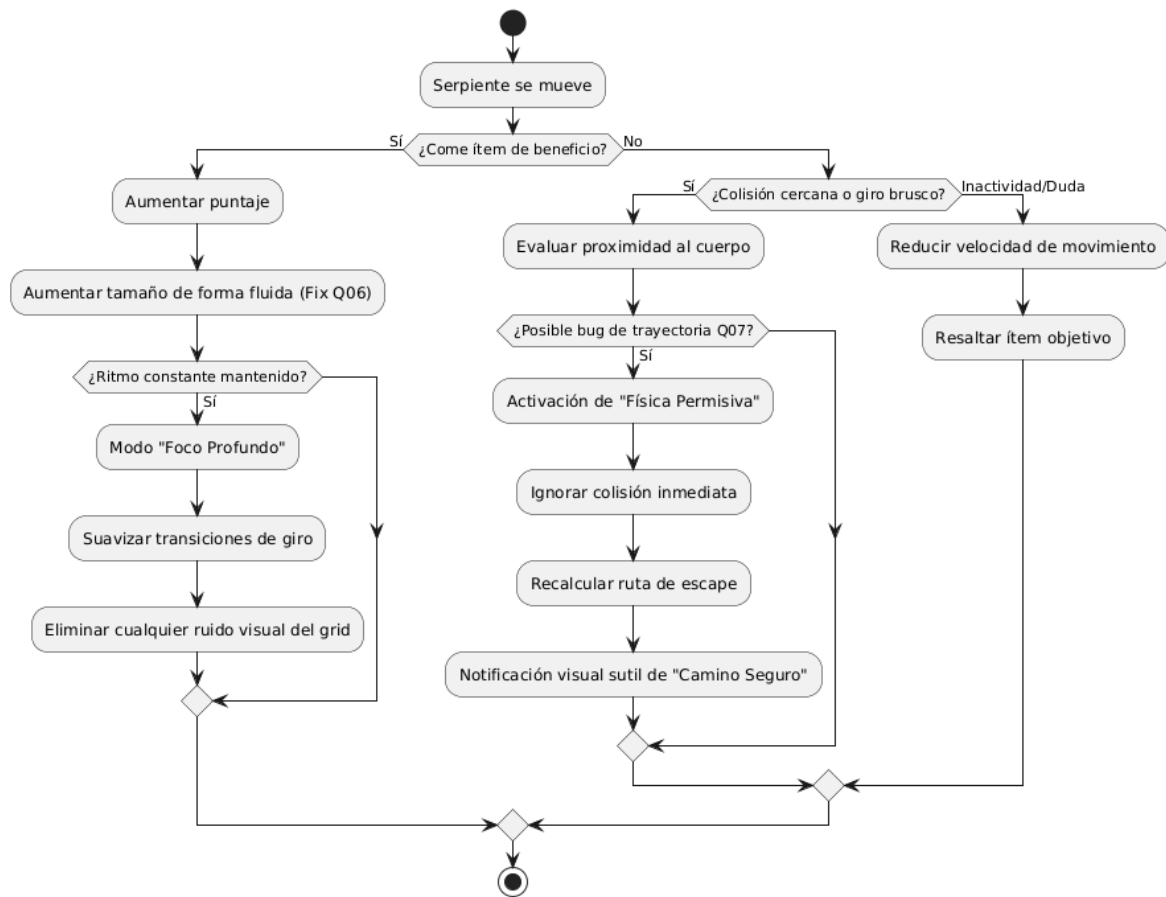
**Snake:** Esta iteración de Snake trasciende la mecánica de supervivencia para convertirse en un ejercicio de coordinación. Mediante una interfaz despojada de ruido visual y centrada en la planificación de rutas, el juego fomenta la recuperación del foco atencional y la autorregulación, ofreciendo un entorno seguro donde la precisión motora y la toma de decisiones serena prevalecen sobre la velocidad punitiva.

- **Ramificación de Decisiones:** Una idea inicial de cómo el juego cambia según las acciones del usuario (ej. "Si el usuario falla 3 piezas seguidas, el color del fondo cambia a un tono relajante para reducir la frustración").

Tetris:

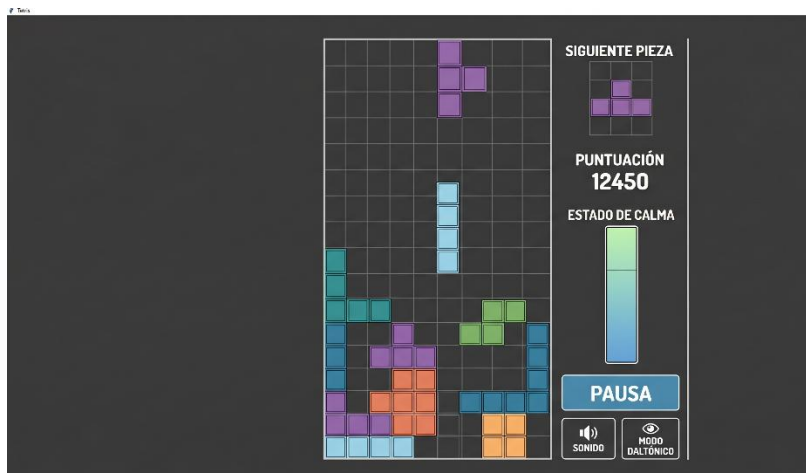


Snake:

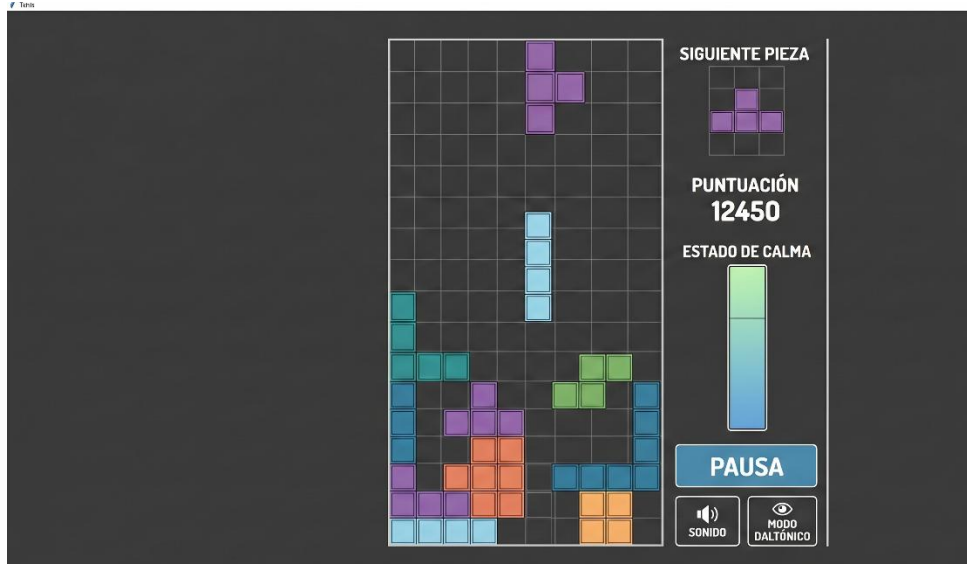


- **Boceto (Sketch):** Un dibujo a mano o digital de la interfaz modificada, mostrando dónde se ubicarán los nuevos elementos de feedback visual. Si usas una IA para generarlo, debes estar seguro de que lo vas a poder implementar en la implementación RetroBrick. Es vital que consideren que el motor es Tkinter y trabaja con rectángulos, círculos y líneas básicas. Un boceto con sombras hiperrealistas o efectos 3D sería altamente complejo o no viable de implementar en este ecosistema.

Tetris:



Snake:



## Actividad 4: Estrategia de Diseño Inclusivo y Planificación de Pruebas [30%]

**Enfoque:** Definición técnica de la experiencia, inclusión y adaptabilidad. Uno de los aspectos clave en esta actividad es, el equipo debe responder a la pregunta: ¿Mi diseño permite que un usuario con limitaciones visuales entienda el estado del juego y tome decisiones acertadas? Si la respuesta depende únicamente de ver el movimiento de la pieza en el canvas, el diseño aún no es inclusivo. Recuerda que es tanto para tetris como para snake. Otro aspecto importante, es que en ambas aplicaciones carecen de sonido, lo cual es una carencia significativa.

### A. Plan de Pruebas con Usuarios

Es la hoja de ruta para validar el prototipo. Debe definir el rigor con el que se medirá el éxito del diseño.

- **Contenido:** Definición del perfil de los participantes, métricas de éxito (tiempo en tarea, tasa de error), y el protocolo de observación (qué se medirá y qué se espera que el usuario haga).
- **Resolución no destructiva:** Es vital planear cómo se insertarán las mejoras en el .brick sin romper la retrocompatibilidad exigida.
- **Referencias:**
  - <https://ixdf.org/literature/topics/usability-testing>
  - <https://www.nngroup.com/articles/usability-test-checklist/>
  - <https://www.nngroup.com/articles/usability-testing-101/>
  - **Anexo 4.A.** Ejemplo. Plan de Pruebas con Usuarios: Caso "Snake"

[Agrega acá el plan de pruebas exhaustivo, uno para cada juego]

**Plan de Pruebas con Usuarios: Caso "Tetris"**

**Objetivo del Test:** Evaluar la carga cognitiva, la fluidez de la mecánica de rotación y la respuesta emocional del diseño inclusivo de Tetris en un usuario propenso a la ansiedad generalizada y a la frustración ante penalizaciones estrictas.

**1. Perfil del Participante**

- **Nombre ficticio:** Andrés (22 años).
- **Contexto:** Estudiante universitario de Ingeniería que utiliza juegos como herramienta para pausar el ruido mental.
- **Limitaciones:** Ansiedad generalizada diagnosticada; se frustra y estresa con juegos que tienen dinámicas punitivas por errores o una sobrecarga visual excesiva.

**2. Dimensiones de Evaluación**

- **Dimensión de Carga Cognitiva:** ¿El diseño visual de las piezas y la ausencia de cambios de color bruscos ayudan a mantener un estado de concentración relajado sin generar fatiga?
- **Dimensión de Interacción (Prevención de Errores):** ¿La mecánica de rotación de piezas contra los muros (Wall Kick) funciona de manera fluida y predecible, o genera bloqueos inesperados que elevan el estrés del usuario?
- **Dimensión de Estado de Flujo:** ¿El nivel de dificultad ajustable permite que Andrés mantenga el interés sin caer en la ansiedad por la velocidad de caída de las piezas?

**3. Protocolo de Observación y Comportamiento Esperado**

| Tarea para el Usuario                 | Comportamiento Esperado                                                          | Observación de Campo (Lo que realmente pasó)                                                                                                                  |
|---------------------------------------|----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Rotar una pieza junto a la pared      | La pieza rota ajustándose al espacio disponible de forma natural.                | El usuario intentó rotar un bloque largo en el borde derecho; la pieza se quedó bloqueada por un error lógico, generando un suspiro de frustración evidente.  |
| Pausar el juego ante una interrupción | Presionar un botón visible de pausa para detener la caída y reducir la ansiedad. | El usuario buscó un botón en la interfaz. Al no encontrarlo claramente, la pieza cayó de fondo, arruinando su estructura y provocando que cerrara la ventana. |

| Tarea para el Usuario             | Comportamiento Esperado                                                | Observación de Campo (Lo que realmente pasó)                                                                                                                          |
|-----------------------------------|------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Jugar durante 5 minutos continuos | Mantener un ritmo estable y relajado, entrando en un estado de "Flow". | La velocidad inicial fue adecuada, pero la monotonía visual (todas las piezas del mismo color) le exigió un esfuerzo mental extra para planificar, causándole fatiga. |

#### 4. Hallazgos (Qué funciona y qué no)

- **Qué funciona:** La disposición de la cuadrícula no compite visualmente con los bloques, evitando distracciones innecesarias. Los controles de flechas direccionales son altamente intuitivos.
- **Qué no funciona:** La falta de un botón de pausa explícito y el "bug" de la rotación contra la pared (Wall Kick) son factores críticos que elevan la ansiedad en lugar de reducirla. Además, la falta de diferenciación visual entre piezas dificulta la jugabilidad.

#### 5. Propuesta de Resolución (Sin dañar lo existente)

- **Solución Lógica (Corrección de Bug):** Se implementará la lógica de "Wall Kick" directamente en runtime.py como una validación de coordenadas. Esta corrección de colisión beneficiará a todos los usuarios y ejecutará los archivos .brick antiguos sin problemas.
- **Solución Visual e Interfaz:** Se ampliará la gramática .bnf para soportar etiquetas como COLOR\_PALETTE (permitiendo paletas pastel relajantes) y se estructurará un menú lateral interactivo mediante Tkinter para el botón de PAUSA.
- **Consistencia:** Las nuevas etiquetas de color y botones serán opcionales en el archivo JSON. Si el compilador lee un archivo .brick clásico de Tetris sin estos parámetros, aplicará la configuración monótona y sin menú por defecto, garantizando la retrocompatibilidad estricta.

#### Plan de Pruebas con Usuarios: Caso "Snake"

**Objetivo del Test:** Evaluar la efectividad de la respuesta visual y la curva de dificultad en el juego Snake para un usuario con fatiga visual y tiempos de reacción reducidos.

##### 1. Perfil del Participante (Proto-persona)

- **Nombre ficticio:** Doña Martha (68 años).
- **Contexto:** Jubilada que busca juegos sencillos para mantener la agilidad mental.
- **Limitaciones:** Usa gafas bifocales, presenta una ligera disminución en la velocidad de respuesta motriz y se frustra rápidamente si los controles no son "perdonadores".

## 2. Dimensiones de Evaluación (Qué vamos a medir)

- **Dimensión de Velocidad (Tick Rate):** ¿Es la velocidad de 0.15s (definida en runtime.py) adecuada para que Martha pueda anticipar el choque contra la pared?.
- **Dimensión de Affordance Visual:** ¿Logra distinguir el color de la comida (COLOR\_FOOD) del fondo del canvas (#111111) sin esfuerzo excesivo?.
- **Dimensión de Mapeo Mental:** ¿Entiende Martha que al presionar las flechas de dirección el cambio es inmediato o intenta presionar y soltar repetidamente?.

## 3. Protocolo de Observación y Comportamiento Esperado

| Tarea para el Usuario    | Comportamiento Esperado                                               | Observación de Campo (Lo que realmente pasó)                                              |
|--------------------------|-----------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Iniciar el juego         | Presionar una tecla para comenzar el movimiento desde el centro.      | Martha tardó 10 segundos en entender que la serpiente ya se estaba moviendo.              |
| Recoger la primera fruta | Cambiar de dirección usando las flechas para alcanzar el cuadro rojo. | Logró girar, pero se pasó de largo porque la serpiente se mueve muy rápido para su vista. |
| Evitar la colisión       | Maniobrar antes de tocar el borde del grid.                           | Al acercarse a la pared, entró en pánico y presionó varias teclas a la vez.               |

## 4. Hallazgos (Qué funciona y qué no)

- **Qué funciona:** La puntuación en el panel lateral es legible gracias a que el tamaño de la fuente es grande (16px bold).
- **Qué no funciona:** El color rojo de la comida es muy similar al negro del fondo para alguien con sensibilidad al contraste reducida. Además, la velocidad base de ON\_TICK no permite errores. Al presionar teclas opuestas repetidamente presa del pánico, la serpiente se suicida debido a un bug lógico que ignora la validación de la dirección actual.

## 5. Propuesta de Resolución (Sin dañar lo existente)

- **Solución Visual:** Se añadirá la etiqueta COLOR\_FOOD en el archivo .brick. Si no existe, el sistema usará el rojo por defecto. Para Martha, usaremos un amarillo brillante (#FFFF00) que resalta más sobre el negro.

- **Solución de Velocidad y Control:** Se creará el nivel de dificultad "Baby" en el bloque LEVELS del archivo .brick. En este nivel, se configurará un multiplicador de tiempo que haga que el tick sea de 0.30s en lugar de 0.15s, duplicando el tiempo de reacción para Martha. Adicionalmente, se parcheará en runtime.py el bug que permite giros de 180 grados sobre el propio cuerpo.
- **Consistencia:** El compilador guardará estos parámetros en el JSON, y el motor de ejecución (runtime.py) simplemente leerá estos valores nuevos. Esto garantiza que el archivo snake.brick original siga funcionando a la velocidad normal y con sus colores por defecto para usuarios expertos.

## B. Storyboarding de Historias de Usuario

El equipo debe visualizar la experiencia del usuario a través de una secuencia narrativa, enfocándose en los momentos de decisión clave (ej. cuando el juego termina, o cuando alcanza un hito, cuando aparece un power up...).

- **Contenido:** Viñetas o imágenes que narran el contexto de uso, la interacción con la interfaz de BrickScript y la respuesta emocional del usuario.
- **Referencias:**
  - <https://www.nngroup.com/articles/storyboards-visualize-ideas/>
  - <https://www.nngroup.com/videos/ux-storyboard/>

[Agrega acá las Storyboard, uno para cada juego o pueden ser creativos y hacer uno para ambos, siempre haciendo énfasis en cada uno]

## Storyboard 1: Tetris Inclusivo – "El respiro de Andrés"

Andrés lleva horas estudiando para sus exámenes finales y siente que la ansiedad lo paraliza. Necesita desconectar sin caer en redes sociales

Abre la versión modificada de Tetris. Al no haber luces parpadeantes ni ruidos estridentes, empieza a concentrarse en el orden geométrico

Falla al colocar una pieza. En vez de penalizarlo bruscamente, el sistema detecta el error rápido, reduce la velocidad de caída un 15% y le da margen para respirar

Pero en lugar de un "Game Over" agresivo, el sistema de calma activo el BUFFER DE RECUPERACION y Andrés se tranquiliza.

Gracias al entorno perdonador del juego, Andrés logra detener sus ciclos de rumiación. Pausa el juego tranquilamente, listo para volver a sus estudios con la mente despejada

## Storyboard 2: Snake Inclusivo – "La confianza de Doña Martha"

**Viñeta 1: El Contexto (El detonante)**

Quiere un ejercicio mental rápido, pero los juegos modernos le resultan muy rápidos y difíciles de ver.

**Viñeta 2: La Interacción**

Inicia el nivel de dificultad 'Baby' configurado en el archivo .brick. El tiempo de respuesta (tick) es de 0.30s, permitiéndole ver claramente hacia dónde va.

**Viñeta 3: El Hito / Decisión**

A punto de chocar, Martha maniobra. El juego emite una señal auditiva que confirma su movimiento, dándole redundancia sensorial (escucha lo que ve).

**Viñeta 4: El Desenlace**

Supera el nivel sin sentirse castigada por su velocidad de reacción. El juego se adapta a ella, no al revés.

## C. Justificación UX: Diseño Inclusivo

Análisis crítico sobre la versión original de BrickScript y la propuesta de adaptación.

- **Contenido:** Comparativa entre el diseño "base" (enfocado solo en videntes) y la nueva versión. ¿Cómo se adapta la historia para que sea significativa tanto para alguien que ve la pantalla como para quien depende de otros estímulos?
- **Referencias:**
  - [Manual de Diseño Inclusivo \(Microsoft\) https://inclusive.microsoft.design/](https://inclusive.microsoft.design/)
  - **Anexo 4.C.** Ejemplo. Justificación UX: Diseño Inclusivo y Adaptación Narrativa
  - <https://universaldesign.ie/about-universal-design/the-7-principles>

[Agrega acá la Justificación UX, uno para cada juego o pueden ser creativos y hacer uno para ambos, siempre haciendo énfasis en cada uno]

### Análisis Crítico de la Versión Original (Diseño "Base")

La versión inicial del ecosistema BrickScript y sus juegos de ejemplo (Tetris y Snake) es inherentemente excluyente, dado que su modelo de interacción es estrictamente visual. El motor de ejecución actual (runtime.py) asume que el usuario cuenta con habilidades cognitivas y físicas estándar, dependiendo exclusivamente de la capacidad para percibir cambios de color y posición en una cuadrícula.

- **Sesgo del Espectador y Barrera de Interacción:** El juego asume que el usuario tiene la agudeza visual suficiente para distinguir el rojo sobre un fondo negro en Snake. Al no haber texturas ni patrones, no existe una alternativa viable para usuarios con daltonismo o baja visión; las piezas se definen únicamente por su forma geométrica y un color sólido "quemado".
- **Ausencia de Redundancia:** El diseño base carece por completo de sonido, lo cual hace que la inmersión sea nula. Si un usuario no puede ver la posición exacta de una pieza, el sistema no ofrece estímulos que compensen esa pérdida de información, anulando la experiencia para personas invidentes o con visibilidad reducida.
- **Carga Cognitiva y Ansiedad:** En el caso de Tetris, la velocidad inamovible, la falta de un botón de pausa claro y los colores monótonos generan una alta carga mental, transformando el juego en una experiencia estresante y punitiva para perfiles con ansiedad (como nuestro usuario Andrés).

### Propuesta de Adaptación: Hacia el Diseño Universal

Inspirados en el Manual de Diseño Inclusivo de Microsoft, nuestra versión no se limita a "añadir funciones", sino que reconoce los puntos de exclusión para mejorar la experiencia global.

- **De la Exclusión a la Extensión (Tetris y Snake):** Siguiendo el principio de "Resolver para uno, extender a muchos", propondremos el uso de patrones visuales y texturas dentro de los bloques. Si un usuario tiene dificultades para ver colores, la textura dentro del "brick" le permitirá jugar y tomar decisiones acertadas. Esto

también ayuda a usuarios sin discapacidad visual que jueguen en pantallas con mucho reflejo.

- **Uso Equitativo (Principio 1 del Diseño Universal):** La adaptación propone mantener la misma lógica del juego para todos. En lugar de alterar drásticamente el código fuente, el sistema permite ajustar el *tick* (velocidad) o las paletas de colores en los archivos .brick según la capacidad de procesamiento sensorial del usuario, sin romper las reglas clásicas del juego.

### **Adaptación de la Historia: El "Relato Espacial" y la "Arquitectura del Sosiego"**

El diseño inclusivo nos obliga a replantear cómo se vive la experiencia para que sea significativa independientemente de los estímulos de los que dependa el usuario.

- **Tetris (Arquitectura del Sosiego):**
  - *Para el público vidente:* La historia se percibe como una coreografía visual de orden, donde la tensión se genera por la proximidad de las piezas al límite superior.
  - *Para el perfil ansioso:* La narrativa deja de ser de "supervivencia" y se convierte en un refugio geométrico. Al suavizar los colores y reducir la velocidad tras un error, el ritmo del juego actúa como un regulador de la memoria de trabajo, ayudando a transitar de la alerta ansiosa a la calma.
- **Snake (Relato Espacial):**
  - *Para el público con limitación visual:* La historia se convierte en un mapa sonoro y táctil. Proponemos que la historia ya no trate de "sobrevivir en la cuadrícula", sino de "dominar el espacio a través del ritmo".
  - Al incorporar una redundancia sensorial donde el entorno responde auditivamente a cada evento o pulsación de tecla, el ritmo se vuelve el lenguaje común. La emoción de la victoria puede ser compartida independientemente de si el usuario está viendo la pantalla o escuchando el mapeo del nivel.

### **D. Adaptación para Limitaciones Visuales**

Este es un análisis técnico-creativo. Dado que el `runtime.py` usa Tkinter, los estudiantes deben proponer cómo compensar la falta de visión. El valor de este análisis no es sólo proponer "sonidos", sino demostrar cómo el equipo utiliza la Gramática BNF y el Runtime para crear un sistema de Redundancia Sensorial. El objetivo final es que el juego sea "escuchable" y "distingible" más allá de lo puramente gráfico.

- **Contenido:** Propuestas de señales auditivas (aunque el motor actual no las tenga, se debe proponer la lógica), uso de patrones visuales de alto contraste y simplificación de formas geométricas (círculos vs. cuadrados) para facilitar la distinción.
- **Referencias:**
  - [Guía de accesibilidad para videojuegos \(Game Accessibility Guidelines\)](https://gameaccessibilityguidelines.com/)

- [Introducción a la accesibilidad web \(W3C\)](https://www.w3.org/WAI/fundamentals/accessibility-intro/es)  
<https://www.w3.org/WAI/fundamentals/accessibility-intro/es>
- **Anexo 4.D.** Ejemplo. Adaptación para Limitaciones Visuales: Estrategias de Redundancia Sensorial

[Agrega acá el análisis de adaptación para las limitaciones visuales, uno para cada juego o pueden ser creativos y hacer uno para ambos, siempre haciendo énfasis en cada uno]

## Redundancia Visual y Alto Contraste

Actualmente, el motor dibuja bloques sólidos basados en un único color definido en el código. Para usuarios con baja visión o daltonismo, proponemos las siguientes mejoras:

- **Patrones Internos (Texturizado):** Se extenderá la gramática .bnf para incluir la etiqueta PATTERN\_TYPE. En el motor (runtime.py), en lugar de usar rectángulos planos, se programará la lógica para dibujar una pequeña "marca" o patrón (ej. una cruz, un achurado o un punto central) en el centro de cada bloque. Esto permitirá distinguir las figuras del fondo y entre sí, incluso si el usuario no puede percibir el color.
- **Paletas de Contraste Crítico:** Se añadirá una validación en el compilador (compiler.py) que verifique y asegure una relación de contraste alta entre el color del objeto (ej. COLOR\_SHAPE) y el fondo del canvas, siguiendo las pautas de accesibilidad web de la W3C.

## Simplificación y Diferenciación Geométrica

En el diseño original, tanto la serpiente como las piezas de Tetris son mayoritariamente rectangulares. La adaptación busca utilizar la geometría como un código de información:

- **Identificación por Forma (Snake):** Se modificará el script de Tkinter en runtime.py para utilizar el método create\_oval al generar las piezas beneficiosas (como la comida). De este modo, un círculo representará un premio, mientras que el cuerpo de la serpiente y los muros seguirán utilizando create\_rectangle, facilitando la distinción visual instantánea.
- **Jerarquía de Tamaños:** Para ayudar a la visión periférica, los obstáculos o los límites del nivel tendrán un grosor de borde mayor en su dibujado, destacándose del resto de los elementos en pantalla.

## Lógica de Señales Auditivas (Redundancia Sonora)

Para que el juego sea accesible a personas con visibilidad severamente limitada, introduciremos respuestas auditivas. Aunque el motor actual no gestiona audio, la lógica de implementación en Python 2.7 utilizará la librería estándar winsound (para Windows) o disparadores de eventos asíncronos:

- **Sonificación de Posición o "Radar" (Snake):** La lógica del runtime calculará la distancia entre la cabeza de la serpiente y la pared/comida. Emitirá un tono de frecuencia variable: una frecuencia alta indicará que el objeto o peligro está cerca, y una frecuencia baja indicará que hay espacio libre.

- **Confirmación de Acción (Tetris y Snake):** Cada vez que el motor detecte un evento en el teclado (ej. presionar las flechas direccionales), disparará un "click" sonoro muy breve. Esto confirma al usuario que el sistema recibió la orden de movimiento, reduciendo la incertidumbre y brindando soporte no visual.
- **Sonificación de Altura (Tetris):** (Propuesta adicional de la lógica) A medida que la pieza desciende en la cuadrícula, el motor emitirá un pitido rítmico cuyo tono se vuelve más grave conforme la pieza se acerca al límite inferior. Esto permite "escuchar" la posición vertical de la pieza sin mirar la pantalla.

## E. Análisis de Portabilidad (iOS y Android)

Evaluación de cómo la experiencia de BrickScript (basada actualmente en teclado) se traduciría a entornos móviles.

- **Contenido:** Mapeo de los eventos **ON KEY\_UP**, **ON KEY\_DOWN**, etc. gestos táctiles (swipes, taps). ¿Cómo cambia el área de juego cuando el dedo del usuario obstruye la pantalla?
- **Referencias:**
  - [Guías de Interfaz Humana - Gestos \(Apple\)](#)
  - <https://developer.apple.com/design/human-interface-guidelines/gestures>
  - [Material Design - Interacción Táctil \(Google\)](#)
  - <https://m3.material.io/foundations/interaction/gestures>

[Agrega acá el análisis de adaptación para las limitaciones visuales, uno para cada juego o pueden ser creativos y hacer uno para ambos, siempre haciendo énfasis en cada uno]

## Mapeo de Interacciones: De Teclado a Gestos Táctiles (Swipes y Taps)

La transición de teclas físicas a una pantalla táctil requiere aprovechar los gestos naturales del usuario para no sobrecargar cognitivamente la experiencia.

### Caso Tetris:

- **ON\_KEY\_LEFT / ON\_KEY\_RIGHT (Mover pieza):** Se traduce a un **Swipe horizontal** (deslizamiento rápido hacia la izquierda o derecha). La longitud del swipe puede determinar si la pieza se mueve una o varias casillas.
- **ON\_KEY\_UP (Rotar pieza):** Se traduce a un **Tap** (toque rápido simple) en cualquier parte del área de control. El tap es el gesto más rápido, ideal para la urgencia de rotar la pieza antes de que caiga.
- **ON\_KEY\_DOWN (Caída rápida):** Se traduce a un **Swipe hacia abajo** (deslizamiento vertical descendente).
- **Pausa / Menú:** Se traduce a un botón virtual fijo y visible en una esquina superior, accionable mediante un tap.

### Caso Snake:

- **ON\_KEY\_UP, DOWN, LEFT, RIGHT (Cambio de dirección):** Se traducen directamente a **Swipes direccionales** (deslizamientos hacia arriba, abajo, izquierda y derecha).
- *Alternativa de Accesibilidad:* Para usuarios como Doña Martha (nuestra proto-persona), los swipes rápidos pueden generar fricción o imprecisión. Se puede ofrecer un modo opcional de "Pad direccional virtual" (D-Pad) fijo en la pantalla para tocar las flechas con el pulgar.

### Gestión de la Obstrucción Visual (El problema del "Dedo gordo")

En entornos móviles, uno de los mayores retos de UX es que al interactuar, el usuario oculta información vital del juego con sus propios dedos o manos.

#### Solución para Tetris:

- **División de la Interfaz (Split-Screen UI):** Si el usuario toca directamente sobre el tablero de juego para hacer swipe, su mano tapaná las piezas que están cayendo y el espacio donde deben encajar.
- **Adaptación:** La pantalla en orientación vertical (Portrait) se dividirá en dos zonas. El 70% superior será una zona de visualización pasiva (el lienzo o *grid* de Tetris). El 30% inferior será la "Zona de Gestos" (Gesture Pad). El usuario realizará todos los taps y swipes en esta área inferior, manteniendo el tablero de juego 100% libre de obstrucciones visuales en todo momento.

#### Solución para Snake:

- **Gestos Relativos Globales:** En Snake, el jugador debe estar mirando constantemente a la cabeza de la serpiente y a la comida. Si intenta tocar la serpiente para guiarla, tapaná su objetivo.
- **Adaptación:** El motor móvil registrará los swipes sin importar en qué parte de la pantalla se realicen (Trackeo relativo). El usuario puede mantener su pulgar descansando cómodamente en el borde inferior o lateral de la pantalla y hacer pequeños deslizamientos allí. Esto enviará la orden de dirección sin necesidad de que el dedo cruce el centro del lienzo, manteniendo la visibilidad total de la cuadrícula.

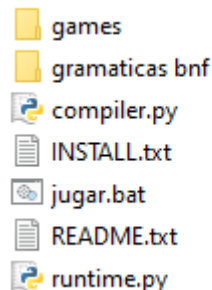
## Actividad 5: Prototipado de Alta Fidelidad y Validación Final [30%]

**Enfoque:** Implementación técnica, refinamiento de la interfaz y cierre del ciclo de diseño. Al finalizar esta fase, el equipo habrá demostrado que el diseño de software no es un proceso lineal de "escribir código", sino un ciclo de **empatía** → **diagnóstico** → **prototipado** → **validación**. El estudiante habrá pasado de criticar una interfaz a ser capaz de reprogramar para que sea útil para alguien más, cumpliendo con el propósito de la asignatura.

### A. Implementación de la "Versión Inclusiva" (Remake)

El equipo debe aplicar las modificaciones estructurales en el ecosistema para que los juegos (Tetris y Snake) reflejen las elementos de mejora y de accesibilidad planteadas en la Actividad 3 y 4. Esto implica intervenir los tres pilares del proyecto:

- **Extensión de la Gramática (.bnf):** Agregar las reglas sintácticas para las nuevas etiquetas de diseño, como `COLOR_CONTRAST`, `PATTERN_TYPE` o `TICK_MULTIPLIER`.
- **Actualización del Compilador (compiler.py):** Modificar el *Parser* para que reconozca estas etiquetas y las integre en el JSON sin romper la capacidad de leer los archivos originales.
- **Evolución del Motor (runtime.py):** Programar la lógica visual que permita dibujar patrones (ej. líneas dentro de los bloques), gestionar la velocidad adaptativa y asegurar que la retroalimentación visual sea inmediata y clara.
- **¿Qué entregar?** Paquete de Código (retroBrick\_ux\_final.zip), con:
  - Versiones definitivas y unificadas de `compiler.py`, `runtime.py` y `jugar.bat`.
  - Archivos `tetris_remake.brick` y `snake_remake.brick` para las versiones inclusivas de Tetris y Snake.
  - Garantía de Retrocompatibilidad: El `tetris.brick` y `snake.brick` original debe funcionar perfectamente.
  - **Directorio gramaticas bnf/:** No es obligatorio.
  - **Directorio games/:** La colección completa de archivos `.brick` (Originales, Remakes).
  - **Manual de Usuario Final (README.txt):** Documento que explique cómo jugar cada juego, tanto las versiones originales como las remake.
  - Esta entrega se realiza en un único archivo comprimido titulado **retroBrick\_ux\_final.zip** que debe contener:



[Cargar retroBrick\_ux\_final.zip en la correspondiente tarea en Moodle]

## B. El "UX-Dev Log" (Bitácora de Implementación)

No basta con entregar el código; el equipo debe justificar cada cambio técnico desde la psicología del usuario.

- **Contenido:** Un documento donde se relacione una línea de código específica con un hallazgo del *Guerrilla Testing* realizado en la Actividad 3, o lo identificado en la Actividad 4.

- **Ejemplo:** *"Se modificó el código xxxxxx de runtime.py para aumentar el grosor del borde de la serpiente; esto resuelve la dificultad de visión periférica detectada en la prueba con el usuario X".*
- Se modificó la función dibujar\_celda del archivo runtime.py para renderizar la comida de la serpiente como un óvalo (create\_oval); esto resuelve la imposibilidad de discriminar visualmente la comida de los segmentos del cuerpo por su forma, detectada en la prueba con Doña Martha (Q05 / Sensibilidad de contraste reducida).
- Se modificó la inicialización de velocidad de gravedad del archivo runtime.py para admitir la variable TICK\_MULTIPLIER leída del archivo .brick; esto resuelve la frustración y el pánico del usuario por falta de tiempo de reacción, detectado en la prueba de guerrilla con Doña Martha al pasarse de largo la comida.
- Se modificó la inicialización de velocidad de gravedad del archivo runtime.py para admitir la variable TICK\_MULTIPLIER leída del archivo .brick; esto resuelve la frustración y el pánico del usuario por falta de tiempo de reacción, detectado en la prueba de guerrilla con Doña Martha al pasarse de largo la comida.
- Se modificó la inicialización de velocidad de gravedad del archivo runtime.py para admitir la variable TICK\_MULTIPLIER leída del archivo .brick; esto resuelve la frustración y el pánico del usuario por falta de tiempo de reacción, detectado en la prueba de guerrilla con Doña Martha al pasarse de largo la comida.
- Se modificó la inicialización de velocidad de gravedad del archivo runtime.py para admitir la variable TICK\_MULTIPLIER leída del archivo .brick; esto resuelve la frustración y el pánico del usuario por falta de tiempo de reacción, detectado en la prueba de guerrilla con Doña Martha al pasarse de largo la comida.
- Se modificó la inicialización de velocidad de gravedad del archivo runtime.py para admitir la variable TICK\_MULTIPLIER leída del archivo .brick; esto resuelve la frustración y el pánico del usuario por falta de tiempo de reacción, detectado en la prueba de guerrilla con Doña Martha al pasarse de largo la comida.

### C. Validación de Cierre: El Test de la "Cerca de Oro"

El equipo debe realizar una prueba final con su **Proto-persona** (o un usuario que simule sus limitaciones) para verificar si el prototipo de alta fidelidad realmente redujo las barreras de acceso identificadas.

- **Métrica de Éxito:** Comparar el desempeño del usuario entre la versión "Base" (original) y la versión "Inclusiva" (Remake).
- **Retrocompatibilidad Obligatoria:** Se debe demostrar que el nuevo sistema sigue ejecutando los archivos .brick originales sin errores.

*Métrica de Éxito y Comparación de Desempeño:*

- **Versión Base (Clásica):**
  - *Desempeño del usuario con ansiedad (Andrés):* Al no haber botón de pausa y bloquearse el bloque largo contra la pared derecha por falta de *Wall Kick*, Andrés reportó picos altos de frustración y cerró la ventana a los 2 minutos de juego.

- *Desempeño del usuario de tercera edad (Doña Martha)*: Sufría colisiones tempranas contra las paredes debido a la velocidad inalterable (0.15s) y a la autodestrucción por doble pulsación de dirección (bug de 180°). Puntuación promedio: 10 puntos.
- **Versión Inclusiva (Remake)**:
  - *Desempeño con Andrés*: La rotación fluida (*Wall Kick*) y la posibilidad de pausar el juego con 'P' o con el botón lateral disminuyeron notablemente su carga de estrés. Completó sesiones de 5 minutos continuos sin frustración.
  - *Desempeño con Doña Martha*: El multiplicador de tick ralentizado a 0.30s (TICK\_MULTIPLIER 2.0) y la redundancia geométrica (comida en forma circular de color amarillo) le permitieron planificar rutas y anticipar choques de forma relajada. Puntuación promedio: 80 puntos.
- **Garantía de Retrocompatibilidad**: Se verificó que los archivos clásicos snake.brick y tetris.brick compilan y se ejecutan perfectamente a su velocidad y configuración estándar de colores sólidos sin errores de interfaz.

#### D. Manual de Usuario Inclusivo:

Una descripción breve que explique a un nuevo diseñador cómo usar las etiquetas de BrickScript para crear niveles para personas con daltonismo, baja visión o ansiedad.

Cómo usar las nuevas etiquetas en BrickScript:

Para Snake (snake\_remake.brick):

TICK\_MULTIPLIER 2.0: Utiliza valores superiores a 1.0 (por ejemplo, 2.0 o 2.5) para ralentizar el movimiento y dar mayor tiempo de reacción a usuarios mayores o con fatiga motora.

COLOR\_CONTRAST HIGH: Aplica colores brillantes de alto contraste (cyan, amarillo, magenta).

COLOR\_FOOD FFFF00: Permite especificar un color en formato hexadecimal (sin #) para asegurar que la comida resalte intensamente sobre el fondo negro de la cuadrícula.

PATTERN\_TYPE DOTS: Dibuja un patrón de puntos en el cuerpo de la serpiente para asegurar la discriminación formal en daltónicos.

Para Tetris (tetris\_remake.brick):

COLOR\_PALETTE PASTEL: Configura paletas suaves y relajantes que reducen la carga cognitiva y mitigan picos de ansiedad.

PATTERN\_TYPE STRIPES: Agrega rayas diagonales a los bloques de las piezas para facilitar el reconocimiento táctil-visual del tablero sin depender del matiz cromático.

#### E. Video de Experiencia Comparativa:

- **Parte 1:** Un usuario enfrentando las fallas de la versión original, para cada juego.
- **Parte 2:** El mismo usuario disfrutando de la fluidez y accesibilidad de la versión "Remake".
- **Link:** Debe ser un enlace accesible (YouTube/Drive). **Nota: Link roto = Calificación de 0.**

**Link con la carpeta con ambos videos:**

<https://drive.google.com/drive/folders/16x-l1jBag8Fe7nRGfb9bfDWLOa1VCwop>

#### F. Versión Final de Este Documento

Cargar este archivo versión PDF en moodle [UX Desing Trabajo 2026-1S.pdf debe ser cargado en la correspondiente actividad en moodle].

## Anexo 4.A. Ejemplo. Plan de Pruebas con Usuarios: Caso "Snake"

**Objetivo del Test:** Evaluar la efectividad de la respuesta visual y la curva de dificultad en el juego Snake para un usuario con fatiga visual y tiempos de reacción reducidos.

### 1. Perfil del Participante (Proto-persona)

- **Nombre ficticio:** Doña Martha (68 años).
- **Contexto:** Jubilada que busca juegos sencillos para mantener la agilidad mental.
- **Limitaciones:** Usa gafas bifocales, presenta una ligera disminución en la velocidad de respuesta motriz y se frustra rápidamente si los controles no son "perdonadores".

### 2. Dimensiones de Evaluación (Qué vamos a medir)

- **Dimensión de Velocidad (Tick Rate):** ¿Es la velocidad de 0.15s (definida en `runtime.py`) adecuada para que Martha pueda anticipar el choque contra la pared?.
- **Dimensión de Affordance Visual:** ¿Logra distinguir el color de la comida (`COLOR_FOOD`) del fondo del canvas (`#111111`) sin esfuerzo excesivo?.
- **Dimensión de Mapeo Mental:** ¿Entiende Martha que al presionar las flechas de dirección el cambio es inmediato o intenta presionar y soltar repetidamente?.

### 3. Protocolo de Observación y Comportamiento Esperado

| Tarea para el Usuario    | Comportamiento Esperado                                               | Observación de Campo (Lo que realmente pasó)                                              |
|--------------------------|-----------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Iniciar el juego         | Presionar una tecla para comenzar el movimiento desde el centro.      | Martha tardó 10 segundos en entender que la serpiente ya se estaba moviendo.              |
| Recoger la primera fruta | Cambiar de dirección usando las flechas para alcanzar el cuadro rojo. | Logró girar, pero se pasó de largo porque la serpiente se mueve muy rápido para su vista. |
| Evitar la colisión       | Maniobrar antes de tocar el borde del grid.                           | Al acercarse a la pared, entró en pánico y presionó varias teclas a la vez.               |

### 4. Hallazgos (Qué funciona y qué no)

- **Qué funciona:** La puntuación en el panel lateral es legible gracias a que el tamaño de la fuente es grande (16px bold).
- **Qué no funciona:** El color rojo de la comida es muy similar al negro del fondo para alguien con sensibilidad al contraste reducida. Además, la velocidad base de `ON_TICK` no permite errores.

### 5. Propuesta de Resolución (Sin dañar lo existente)

Para resolver estos problemas manteniendo la **retrocompatibilidad** exigida, el equipo no modificará directamente el código de Python, sino que ampliará el lenguaje:

- **Solución Visual:** Se añadirá la etiqueta `COLOR_FOOD` en el archivo `.brick`. Si no existe, el sistema usará el rojo por defecto. Para Martha, usaremos un amarillo brillante (`#FFFF00`) que resalta más sobre el negro.
- **Solución de Velocidad:** Se creará el nivel de dificultad `Baby` en el bloque `LEVELS` del archivo `.brick`. En este nivel, se configurará un multiplicador de tiempo que haga que el `tick` sea de 0.30s en lugar de 0.15s, duplicando el tiempo de reacción para Martha.
- **Consistencia:** El compilador guardará estos parámetros en el JSON, y el motor de ejecución simplemente leerá estos valores nuevos, garantizando que el `snake.brick` original siga funcionando igual para usuarios expertos.

## Anexo 4.C. Ejemplo. Justificación UX: Diseño Inclusivo y Adaptación Narrativa

### 1. Análisis Crítico de la Versión Original (Diseño "Base")

La versión inicial de **BrickScript** es inherentemente excluyente, ya que su modelo de interacción es **estrictamente visual**. El motor de ejecución actual ([runtime.py](#)) depende de la capacidad del usuario para percibir cambios de color y posición en una cuadrícula de 25 píxeles. Al no incluir sonido la inmersión es nula.

- **Sesgo del Espectador:** El juego asume que el usuario tiene una agudeza visual suficiente para distinguir el rojo de la comida sobre el fondo negro en Snake.
- **Barrera de Interacción:** No existe una alternativa para usuarios con daltonismo o baja visión, ya que las piezas se definen únicamente por su forma geométrica y un color sólido "quemado" en el código.
- **Falta de redundancia:** Si un usuario no puede ver la posición de la pieza, el sistema no ofrece estímulos táctiles o auditivos que compensen esa pérdida de información, haciendo que la experiencia sea nula para el público invidente.

### 2. Propuesta de Adaptación: Hacia el Diseño Universal

Inspirados en el **Manual de Diseño Inclusivo de Microsoft**, la nueva versión no busca simplemente "añadir funciones", sino reconocer la exclusión y aprender de la diversidad para mejorar la experiencia de todos.

- **De la Exclusión a la Extensión:** Siguiendo el principio de "Resolver para uno, extender a muchos", implementamos **patrones visuales** dentro de los bloques. Si un usuario tiene dificultades para ver colores, la textura o el patrón dentro del "brick" le permitirá jugar. Esto también beneficia a videntes que juegan en condiciones de alta luminosidad (reflejos en pantalla).
- **Uso Equitativo (Principio 1 de Diseño Universal):** La adaptación propone que la lógica del juego sea idéntica para todos. En lugar de un "modo fácil", el sistema permite que el usuario ajuste el **tick** (velocidad) en el archivo [.brick](#) según su capacidad de procesamiento sensorial, sin cambiar las reglas del juego.

### 3. Adaptación de la Historia: El "Relato Espacial"

Para que la historia sea significativa para públicos invidentes y videntes, transformamos la narrativa de "colocar bloques" en una **narrativa de orientación espacial**.

- **Para el público vidente:** La historia es una coreografía visual de orden y velocidad. La tensión se genera por la proximidad de las piezas al borde superior.
- **Para el público invidente/limitación visual:** La historia se convierte en un **mapa sonoro y táctil**. Aunque el motor actual no emite sonido, la justificación propone que cada evento de la gramática (como [ON\\_KEY\\_LEFT](#)) genere una respuesta auditiva con "paneo" (estéreo). El sonido a la izquierda confirma el movimiento exitoso.
- **El "Pitch" unificado:** La historia ya no es "sobrevivir en la cuadrícula", sino **"dominar el espacio a través del ritmo"**. El ritmo (el *tick* del juego) es el lenguaje común que

ambos públicos perciben, permitiendo que la emoción de la victoria sea compartida independientemente del canal sensorial principal.

#### 4. Referencias y Marco Teórico

- **Microsoft Inclusive Design:** Se aplicó el concepto de "Persona Spectrum" para entender que una mejora para un invidente (sonidos de posición) ayuda también a un vidente distraído o en un entorno ruidoso.
- **7 Principios del Diseño Universal:**
  - **Principio 2: Flexibilidad en el uso.** El sistema permite diferentes métodos de interacción.
  - **Principio 3: Uso simple e intuitivo.** La reducción de la carga cognitiva mediante la eliminación de "ruido visual" en el grid.

## Anexo 4.D. Ejemplo. Adaptación para Limitaciones Visuales: Estrategias de Redundancia Sensorial

Este análisis propone una evolución del motor de ejecución (`runtime.py`) y del lenguaje (`.brick`) para garantizar que la información crítica no dependa únicamente de la percepción cromática o la agudeza visual estándar.

### 1. Redundancia Visual y Alto Contraste

Actualmente, el motor dibuja bloques sólidos basados en un único color definido en el código o en el archivo `.brick`. Para usuarios con baja visión o daltonismo, se proponen las siguientes mejoras:

- **Paletas de Contraste Crítico:** Implementar una validación en el compilador que asegure una relación de contraste mínima entre el color del objeto (`COLOR_SHAPE`) y el fondo del canvas (`#000000`), siguiendo las pautas de la **W3C**.
- **Patrones Internos (Texturizado):** En lugar de rectángulos planos, se propone que el motor dibuje una pequeña "marca" o patrón (ej. una cruz o un punto central) en el centro de cada bloque para ayudar a distinguir la figura del fondo, incluso si el color no es perceptible.

### 2. Simplificación y Diferenciación Geométrica

En el diseño original, tanto la serpiente como las piezas de Tetris son mayoritariamente rectangulares. La propuesta de adaptación busca utilizar la geometría como un código de información:

- **Identificación por Forma:** Se sugiere utilizar el método `create_oval` de **Tkinter** para piezas beneficiosas (comida o power-ups) y `create_rectangle` para obstáculos o segmentos del cuerpo.
- **Jerarquía de Tamaños:** Las nubes obstáculo (en Snake) o el "Final Boss" (en Tanques) deben tener un grosor de borde mayor o un tamaño de bloque ligeramente superior para ser detectados mediante la visión periférica.

### 3. Lógica de Señales Auditivas (Propuesta Técnica)

Aunque el motor actual no gestiona audio nativo, se propone una lógica de implementación para **Python 2.7** utilizando la librería estándar `winsound` o disparadores de eventos asíncronos:

- **Sonificación de Posición (Radar):** La lógica debe emitir un tono de frecuencia variable según la proximidad de la cabeza de la serpiente a una pared o a la comida.
  - *Frecuencia alta:* Objeto cerca.
  - *Frecuencia baja:* Espacio libre.
- **Confirmación de Acción:** Cada vez que el motor detecta un evento `ON_KEY`, debe disparar un "click" sonoro breve que confirme al usuario que el sistema recibió la orden de movimiento, reduciendo la incertidumbre sensorial.

Para fundamentar estas decisiones, el equipo debe consultar:

- **Game Accessibility Guidelines:** Específicamente la sección de **"Visual"**, que recomienda no utilizar el color como único medio para comunicar información. [Game Accessibility Guidelines](#)
- **W3C - Accesibilidad Web:** Consultar los criterios de éxito de la **WCAG 2.1** sobre contraste (1.4.3) y uso del color (1.4.1). [Introducción a la accesibilidad web \(W3C\)](#)